

hw2

Yifeng Huang

This is the link to the repo: <https://github.com/YifengHuang-Bob/Homework2>

Problem 1: Random Walk with Momentum

a. Simulation function

```
#' Simulate a two-dimensional random walk with momentum
#'
```

```
#' @param n A positive integer giving the number of steps.
#' @param p A numeric value between 0 and 1 giving the probability of
#' preserving the previous direction.
#' @return A numeric vector containing the final x and y coordinates.
momentum_walk <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) ||
      n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer.")
  }

  if (!is.numeric(p) || length(p) != 1 || is.na(p) ||
      p < 0 || p > 1) {
    stop("p must be a numeric value between 0 and 1.")
  }

  directions <- rbind(
    right = c(1, 0),
    left = c(-1, 0),
    up = c(0, 1),
    down = c(0, -1)
  )

  position <- c(0, 0)

  direction <- sample(1:4, 1)
  position <- position + directions[direction, ]

  if (n > 1) {
    for (i in 2:n) {
```

```

    if (runif(1) >= p) {
      direction <- sample(1:4, 1)
    }

    position <- position + directions[direction, ]
  }
}

return(position)
}

```

```
momentum_walk(10, 0.6)
```

```
[1] -2  0
```

```
momentum_walk(10, 0.6)
```

```
[1] -2 -2
```

b. Implement the simulation

Version 1: Loop implementation

```

#' Simulate a two-dimensional random walk with momentum using a loop
#'
#' @param n A positive integer giving the number of steps.
#' @param p A numeric value between 0 and 1 giving the probability of
#'   preserving the previous direction.
#' @return A numeric vector containing the final x and y coordinates.
momentum_walk1 <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) ||
      n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer.")
  }

  if (!is.numeric(p) || length(p) != 1 || is.na(p) ||
      p < 0 || p > 1) {
    stop("p must be a numeric value between 0 and 1.")
  }
}

```

```

directions <- rbind(
  right = c(1, 0),
  left = c(-1, 0),
  up = c(0, 1),
  down = c(0, -1)
)

position <- c(0, 0)

direction <- sample(1:4, 1)
position <- position + directions[direction, ]

if (n > 1) {
  for (i in 2:n) {
    if (runif(1) >= p) {
      direction <- sample(1:4, 1)
    }

    position <- position + directions[direction, ]
  }
}

return(position)
}

```

Version 2: vapply implementation

```

#' Simulate a two-dimensional random walk with momentum using vapply
#'
#' @param n A positive integer giving the number of steps.
#' @param p A numeric value between 0 and 1 giving the probability of
#' preserving the previous direction.
#' @return A numeric vector containing the final x and y coordinates.
momentum_walk2 <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) ||
      n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer.")
  }

  if (!is.numeric(p) || length(p) != 1 || is.na(p) ||
      p < 0 || p > 1) {
    stop("p must be a numeric value between 0 and 1.")
  }
}

```

```

directions <- rbind(
  right = c(1, 0),
  left = c(-1, 0),
  up = c(0, 1),
  down = c(0, -1)
)

keep_direction <- runif(n - 1) < p

block_id <- c(1L, 1L + cumsum(!keep_direction))
n_blocks <- max(block_id)

block_direction <- vapply(
  seq_len(n_blocks),
  function(i) sample(1:4, 1),
  integer(1)
)

step_direction <- block_direction[block_id]

steps <- directions[step_direction, , drop = FALSE]

return(colSums(steps))
}

```

```
momentum_walk1(10, 0.6)
```

```
[1] 7 -1
```

```
momentum_walk2(10, 0.6)
```

```
[1] 6 2
```

```
momentum_walk1(1000, 0.6)
```

```
[1] -2 -50
```

```
momentum_walk2(1000, 0.6)
```

```
[1] -123 73
```

Version 1 simulates the random walk sequentially using a `for` loop. The first direction is selected uniformly at random. For each subsequent step, the previous direction is preserved with probability (p); otherwise, a new direction is selected uniformly from the four possible directions. The position is updated after every step.

Version 2 first generates the keep/change decisions for steps 2 through (n). These decisions divide the walk into blocks in which the direction remains unchanged. `vapply()` is used to randomly select one direction for each block. The block directions are then expanded to the individual steps, and `colSums()` is used to calculate the final position.

c. Reproducibility

```
set.seed(506)
momentum_walk1(10, 0.6)
```

```
[1] -2 0
```

```
set.seed(506)
momentum_walk1(10, 0.6)
```

```
[1] -2 0
```

```
set.seed(506)
momentum_walk1(1000, 0.6)
```

```
[1] -14 -30
```

```
set.seed(506)
momentum_walk1(1000, 0.6)
```

```
[1] -14 -30
```

I used `set.seed(506)` before each call to `momentum_walk1()`. Setting the same seed resets R's random-number generator to the same starting state. Therefore, the same sequence of random numbers is generated each time, so the simulation produces the same result when the function and its inputs are unchanged.

For ($n = 10$) and ($p = 0.6$), both runs produced the final position $((-2, 0))$. For ($n = 1000$) and ($p = 0.6$), both runs produced the final position $((-14, -30))$, demonstrating that the simulation is reproducible.

d. Does momentum matter?

i. Final positions and distances

```
set.seed(506)

positions <- rbind(
  p_0 = momentum_walk1(1000, 0),
  p_05 = momentum_walk1(1000, 0.5),
  p_09 = momentum_walk1(1000, 0.9)
)

distances <- sqrt(rowSums(positions^2))

results <- data.frame(
  p = c(0, 0.5, 0.9),
  x = positions[, 1],
  y = positions[, 2],
  distance = distances
)

results
```

	p	x	y	distance
p_0	0.0	3	-23	23.19483
p_05	0.5	-62	6	62.28965
p_09	0.9	-25	-155	157.00318

ii. Effect of increasing momentum

The final distances for ($p = 0$), ($p = 0.5$), and ($p = 0.9$) were approximately 23.19, 62.29, and 157.00, respectively. In this simulation, increasing the momentum probability appears to increase the final distance from the starting point.

iii. Reasoning

A higher value of (p) makes the particle more likely to continue moving in the same direction as the previous step. This creates longer runs in the same direction and reduces cancellation between movements in opposite directions, so the particle tends to travel farther from the origin.

e. Monte Carlo simulation

i. Simulate 1,000 walks for each momentum value

```
set.seed(506)

p_values <- c(0, 0.25, 0.5, 0.75, 0.9)

distance_samples <- lapply(p_values, function(p) {
  replicate(1000, {
    position <- momentum_walk2(1000, p)
    sqrt(sum(position^2))
  })
})
```

ii. Calculate the mean final distance

```
mean_distances <- vapply(
  distance_samples,
  mean,
  numeric(1)
)
```

iii. Monte Carlo results

```
monte_carlo_results <- data.frame(
  p = p_values,
  mean_distance = mean_distances
)

monte_carlo_results
```

	p	mean_distance
1	0.00	27.60970
2	0.25	35.82068
3	0.50	47.92998
4	0.75	75.42132
5	0.90	121.74651

iv. Effect of momentum on mean final distance

The mean final distance increases as the momentum probability (p) increases. The estimated mean distance rises from approximately 27.61 when ($p = 0$) to approximately 121.75 when ($p = 0.9$).

This suggests that stronger momentum causes the particle to travel farther from the origin on average. As (p) increases, the particle is more likely to continue moving in the same direction for

consecutive steps, creating longer directional runs and reducing cancellation between opposite movements.

f. Compare computational performance

Benchmark for (n = 1000)

```
library(microbenchmark)

benchmark_1000 <- microbenchmark(
  loop_version = momentum_walk1(1000, 0.6),
  vapply_version = momentum_walk2(1000, 0.6),
  times = 100L,
  unit = "ms"
)

benchmark_1000
```

```
Unit: milliseconds
      expr      min       lq     mean  median       uq      max neval
loop_version 1.330102 1.385151 1.577792 1.437000 1.533351 4.632301   100
vapply_version 0.800901 0.917901 1.016264 0.950601 1.010301 2.630901   100
```

Benchmark for (n = 100000)

```
benchmark_100000 <- microbenchmark(
  loop_version = momentum_walk1(100000, 0.6),
  vapply_version = momentum_walk2(100000, 0.6),
  times = 100L,
  unit = "ms"
)

benchmark_100000
```

```
Unit: milliseconds
      expr      min       lq     mean  median       uq      max neval
loop_version 150.3691 155.3810 159.4845 157.9024 160.5601 195.0757   100
vapply_version  98.1071 102.3036 105.7223 104.4812 106.0953 138.9788   100
```

i. Which implementation is faster?

The `vapply` implementation is faster for both input sizes. For ($n = 1000$), the median execution time was approximately 1.44 ms for the loop version and 0.95 ms for the `vapply` version. For ($n = 100000$), the corresponding median execution times were approximately 157.9 ms and 104.48 ms.

ii. Does the difference become larger for the larger input?

Yes. For ($n = 1000$), the difference in median execution time was approximately 0.49 ms, whereas for ($n = 100000$), the difference was approximately 53.42 ms. Thus, the performance advantage of the `vapply` implementation becomes much more noticeable for the larger input.

iii. Why might the implementations have different execution times?

The loop implementation processes the walk one step at a time in an R `for` loop, repeatedly generating random numbers, checking the momentum condition, and updating the position. In contrast, the `vapply` implementation generates the keep/change decisions at once and performs much of the remaining work using vectorized operations such as `cumsum()`, indexing, and `colSums()`. These operations reduce the amount of step-by-step R-level loop overhead, which can lead to faster execution, especially when the number of steps is large.